DAILY FUSION CODE · WEEK 1

Coil current: a first-order system in Simulink

Simulink | Fri 16 Oct 2026 | ~35 min

A tokamak's poloidal-field coils are driven by power supplies, and to first order each coil is an RL circuit. When the supply switches on a voltage V_0, the coil current obeys

$$L\,\frac{dI}{dt} = V_0 - R\,I, \qquad I(0) = 0$$

You'll build this as a block diagram, then run it with Simulink's Euler solver to see yesterday's stability limit in a new setting.

New to Simulink or time constants? Read the Theory pages first.

Tasks

Parameters are in `params.m`: `L = 0.02` H, `R = 0.1` Ω, `V0 = 50` V, `t_stop = 1.5` s. Run it before simulating (or call it from the model's InitFcn callback, see THEORY).

1. **Build `model.slx`.** Use only these blocks: Step, Sum, Gain, Integrator, To Workspace (plus a Scope if you like).
   - Step: step time `0`, initial value `0`, final value `V0`.
   - Integrator output is `I`, initial condition `0`.
   - Use the variable names `L`, `R`, `V0` in the block parameters, not typed-in numbers.
   - To Workspace: variable name `I_out`, save format **Timeseries**.
   - Stop time `t_stop`.
2. **Accurate run.** Solver: variable-step `ode45`, *Max step size* `0.01`. Run it.
3. **Write `analysis.m`** that:
   - runs `params`, then simulates with `out = sim('model');`
   - plots `I(t)` from Simulink together with the analytic solution (derive it yourself) on one axis,
   - reads off from the simulation: the final current, and the time at which I first reaches 63.2 % of the final current,
   - then re-runs the model with the fixed-step `ode1` solver for step sizes `h = 0.05, 0.3, 0.5` s and prints `I` at `t_stop` for each,
   - saves the figure as `output.png` (second subplot: the three `ode1` runs, use markers so you can see the steps).
4. **Answer in `answers.md`** (a few lines):
   - the measured time constant and how it compares to the formula,
   - what happens for each `h`, and the largest `h` for which `ode1` is stable here. Derive that limit by hand, don't just read it off the plot.

Check yourself

- `ode45` : $I(0.2\ \mathrm{s}) \approx 316\ \mathrm{A}$.
- `ode1 , h = 0.05` : $I(0.2\ \mathrm{s}) \approx 341.8\ \mathrm{A}$.

Submit

Commit and push `model.slx` , `analysis.m` , `output.png` and `answers.md` .

THEORY

First-order systems and your first Simulink model

First-order systems

Many systems relax towards an equilibrium at a rate proportional to how far they are from it:

$$\tau \frac{dy}{dt} = y_\infty - y$$

τ is the **time constant** and y_∞ the final value. Coil currents, heat in a plasma after the heating switches off (energy confinement time τ_E), and a thermocouple tracking a wall temperature all look like this.

For a constant y_∞ and $y(0) = y_0$, separating variables gives

$$y(t) = y_\infty + (y_0 - y_\infty)\, e^{-t/\tau}$$

Useful landmarks: after $t = \tau$ the system has covered $1 - e^{-1} \approx 63.2\,\%$ of the way; after 3τ, 95 %; after 5τ, 99.3 %. Reading off the 63.2 % time is the standard way to *measure* τ from data.

To use this on any linear first-order ODE, divide by the coefficient of y and match terms against the form above: that gives you τ and y_∞ in terms of the physical parameters.

Simulink: an ODE as a loop of blocks

Simulink doesn't solve equations you write down; it integrates signals. The recipe for any ODE:

1. Write it as $\frac{dx}{dt} = (\text{something built from } x \text{ and inputs})$.
2. Put an **Integrator** block in the middle. Its *output* is x; its *input* must be dx/dt.
3. Build dx/dt out of x (fed back from the integrator output) and the inputs, using **Sum** and **Gain** blocks.

Minimal example (not today's model): a falling ball, $\ddot{z} = -g$, $z(0) = h_0$, $\dot{z}(0) = 0$.

```
[Constant: -g] --> [Integrator, IC 0] --v--> [Integrator, IC h0] --z--> [To Workspace: z_out]
```

Two integrators in a row turn acceleration into velocity, then position. A first-order ODE needs one integrator plus a feedback path.

Blocks you need today (Library Browser, or click the canvas and type the name):

Block	Library	What it does
Step	Sources	jumps from *Initial value* to *Final value* at *Step time*
Sum	Math Operations	adds inputs; *List of signs* `+-` gives $u_1 - u_2$
Gain	Math Operations	multiplies by a constant (can be an expression like `1/L`)
Integrator	Continuous	$x(t) = x_0 + \int_0^t u\, dt'$
To Workspace	Sinks	stores the signal in the simulation output

Drag from a block's output port to an input port to connect; to branch an existing line, drag from the line while holding Ctrl (Windows) or right-click-drag. Ctrl+R rotates, Ctrl+I flips a block, which keeps feedback paths tidy.

Parameters and running from a script

Block parameters can be any MATLAB expression in the workspace, e.g. a Gain of `k/m`. If `k` isn't defined when you press Run, you get an "undefined variable" error. Either run your parameter script first, or put its name in *Model Settings → Model Properties → Callbacks → InitFcn* so it runs automatically on every simulation.

From a script:

```
params;                                  % define the variables
out = sim('ball');                       % run ball.slx with its saved settings
plot(out.z_out.Time, out.z_out.Data)     % Timeseries: .Time and .Data
out2 = sim('ball', 'Solver', 'ode1', 'FixedStep', '0.1');  % override solver for one run
```

`sim` returns a `SimulationOutput` object; each To Workspace variable is a field of it. Solver options passed to `sim` are strings.

Solvers: ode45 vs ode1

Simulink advances the integrators with a numerical ODE solver, set in *Model Settings → Solver*.

- **Variable-step `ode45`** (default): a 4th/5th-order Runge–Kutta method that picks its own step to meet a tolerance. Accurate, but it may take few, large steps on smooth curves, so plots look jagged. *Max step size* caps the step.
- **Fixed-step `ode1`**: plain explicit Euler with step h, exactly the method from the tritium exercise. Everything there about order and stability applies here.

Refresher on stability. For $dy/dt = -\lambda y$, one Euler step multiplies y by $(1 - \lambda h)$. For $\tau\, dy/dt = y_\infty - y$, the deviation $y - y_\infty$ obeys exactly such an equation. Work out what plays the role of λ, and the tritium THEORY tells you where oscillation and blow-up start.

Tips and pitfalls

- **Step block default step time is 1 s**, not 0. Forgetting to change it shifts your whole response by a second.
- **Sum block sign order** matches the input ports top to bottom. Check which port gets the feedback.
- **Jagged ode45 plot?** That's the solver taking big steps, not an error. Set *Max step size* or *Refine factor*.
- **Units:** H/Ω = s. Check that your τ comes out in seconds and your current in amperes before trusting the plot.